

第2章 Python序列

学习主要内容

- 1.列表对象list的创建与删除
- 2.列表对象方法应用

2.0 处理数据

- 请思考日常生活中的数据
 - 联系人
 - 邮件
 - 日历
 - 代办事宜
 - 选课名单
 - 成绩
 - 采购清单
 - 一副/多副扑克牌

2.0 序列(sequence)

- **序列**：是一系列值（对象），它们通常是相关的，以某种规则进行排列。
- 序列中每个组成部分称为“**元素**” (element)。
- 序列是程序设计中经常用到的数据存储方式，几乎每一种程序设计语言都提供了表格数据结构，如C和Basic中的一维、多维数组等。
- Python提供的序列类型在所有程序设计语言中是最丰富，最灵活，也是功能最强大的。

2.0 序列

- Python中常用的序列结构有列表、元组、字典*、字符串、集合*以及range等等。

	可变	不可变
有序	列表	元组**、字符串
无序	字典、集合	

* 在有些表述中，字典和集合不属于序列。

** 元组的元素（所引用对象）的值可变。

有序序列（列表、元组、字符串）支持双向索引，第一个元素下标为0，第二个元素下标为1，以此类推；最后一个元素下标为-1，倒数第二个元素下标为-2，以此类推。

2.0 序列

```
a_list = ['a', 'b', 'mpilgrim', 'z',  
          'example']
```

```
a_tuple = (element1, element2, ... ,  
            elementn)
```

```
a_dict = {'数学': 95, '英语': 92, '语文':  
          84}
```

```
a_set = {element1, element2, ..., elementn}
```

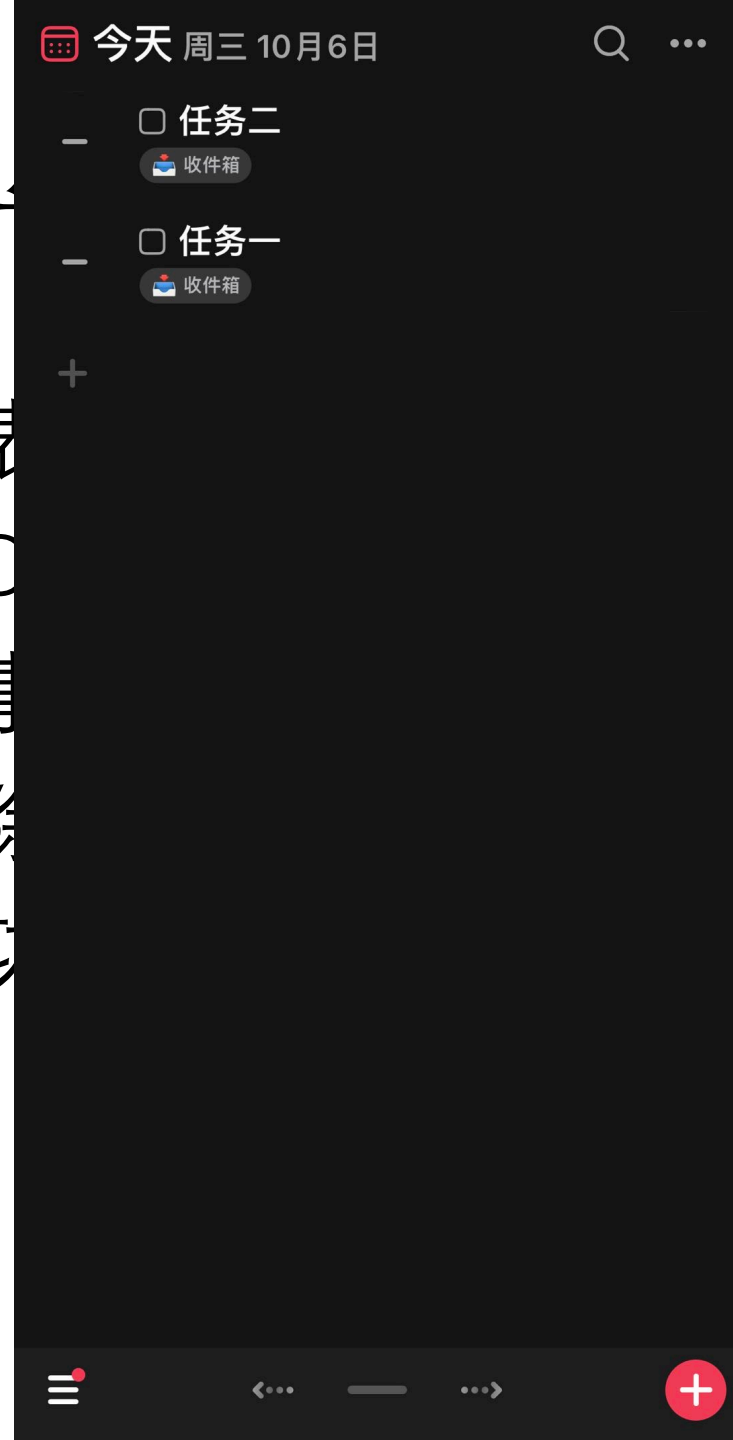
序列的共性方法

函数	功能
<code>len()</code>	计算序列的长度，即返回序列中包含多少个元素。
<code>max()</code>	找出序列中的最大元素。
<code>min()</code>	找出序列中的最小元素。
<code>list()</code>	将序列转换为列表。
<code>str()</code>	将序列转换为字符串。
<code>sum()</code>	计算元素和。
<code>sorted()</code>	对元素进行排序。
<code>reversed()</code>	反向序列中的元素。
<code>enumerate()</code>	将序列组合为一个索引序列，多用在 <code>for</code> 循环中。

注意，对序列使用 `sum()` 函数时，做加和操作的必须都是数字，不能是字符或字符串，否则该函数将抛出异常，因为解释器无法判定是要做连接操作（`+` 运算符可以连接两个序列），还是做加和操作。

课程作业预

- 利用Python的列表
- 完成自己的TODO
- 应该具有已完成事
- 增加，修改，删除
- 能够具备排序的功



之间的转换

2.1 列表

列表

```
suits =  
["♠", "♥", "♣", "♦"]
```

2.1 列表(list)

- 列表是Python中内置可变序列，是若干元素的有序集合。
- 列表中的每一个数据称为元素，列表的所有元素放在一对中括号 “[” 和 “]” 中，并使用逗号分隔开；
- 在Python中，每个元素的数据类型可以各不相同，可以同时分别为整数、实数、字符串等基本类型，甚至是列表、元素、字典、集合以及其他自定义类型的对象。例如：

```
[10, 20, 30, 40]
```

```
['crunchy frog', 'ram bladder', 'lark vomit']
```

```
['spam', 2.0, 5, [10, 20]]
```

```
[['file1', 200,7], ['file2', 260,9]]
```

2.1.1 列表创建与删除

- 如同其他类型的Python对象变量一样，可以使用**字面值literal**来创建对象。
- 在形式上，列表的所有元素放在一对**方括号[]**中，相邻元素之间使用**逗号**分隔。
- 在Python中，**同一个列表中元素的数据类型可以各不相同**，可以同时包含整数、实数、字符串等基本类型的元素，也可以包含列表、元组、字典、集合、函数以及其他任意对象。
- 如果只有一对方括号而没有任何元素则表示空列表。

课堂练习：在SHELL创建列表

```
a_list = ['a', 'b', 'mpilgrim', 'z', 'example']
```

```
a_list = [] #创建空列表
```

```
var_a = 1024
```

```
var_b = "hello"
```

```
list_a = [var_a, var_b]
```

```
list_b = ['spam', 2.0, 5, [10, 20]]
```

```
list_c = [['file1', 200, 7], ['file2', 260, 9]]
```

```
list_d = [{3}, {5:6}, (1, 2, 3)]
```

2.1 列表

- 列表(有序序列)元素的访问
 - 由于列表是有序序列，所以可以通过下标来访问列表中某个元素。a = List[index]
 - 第一个元素下标为0，第二个元素下标为1，以此类推，直到len(list)-1；
 - 最后一个元素下标为-1，倒数第二个元素下标为-2，以此类推，直到-len(list)。
 - 下标 -i 对应于 len(list)-i
 - 如果下标越界，程序会报错(Exception IndexError)。

0	1	2	3	4	[0,5)
10	20	30	40	50	len(list)=5
-5	-4	-3	-2	-1	[-5,0)

索引



0 1 2 3 ... n-1 ← 索引（下标）



-n -(n-1) -(n-2) ... -2 -1 ← 索引（下标）

课堂练习：月份转换

```
monthNames=['January','February','March',  
'April','May','June','July','August',  
'September','October','November','December']
```

```
while(True):  
    month=int(input('请输入月份数:'))  
    print('The month is',monthNames[month])
```

- 1.复制左侧代码至IDLE环境运行,
- 2.请输入任意整数感受效果
- 3.直接回车或者输入其他类型数据感受效果

下标越界错误

请输入月份数:1

The month is February

请输入月份数:2

The month is March

请输入月份数:3

The month is April

请输入月份数:12

Traceback (most recent call last):

File "/Users/bxliu/Documents/1.py", line 8, in <module>
| print('The month is',monthNames[month])

IndexError: list index out of range

二维数组访问

#二维数组的访问：

```
>>>aList = [[1, 2, 3], [4, 5, 6]]
```

```
>>>aList[0]
```

```
[1, 2, 3]
```

```
>>>aList[0][0]
```

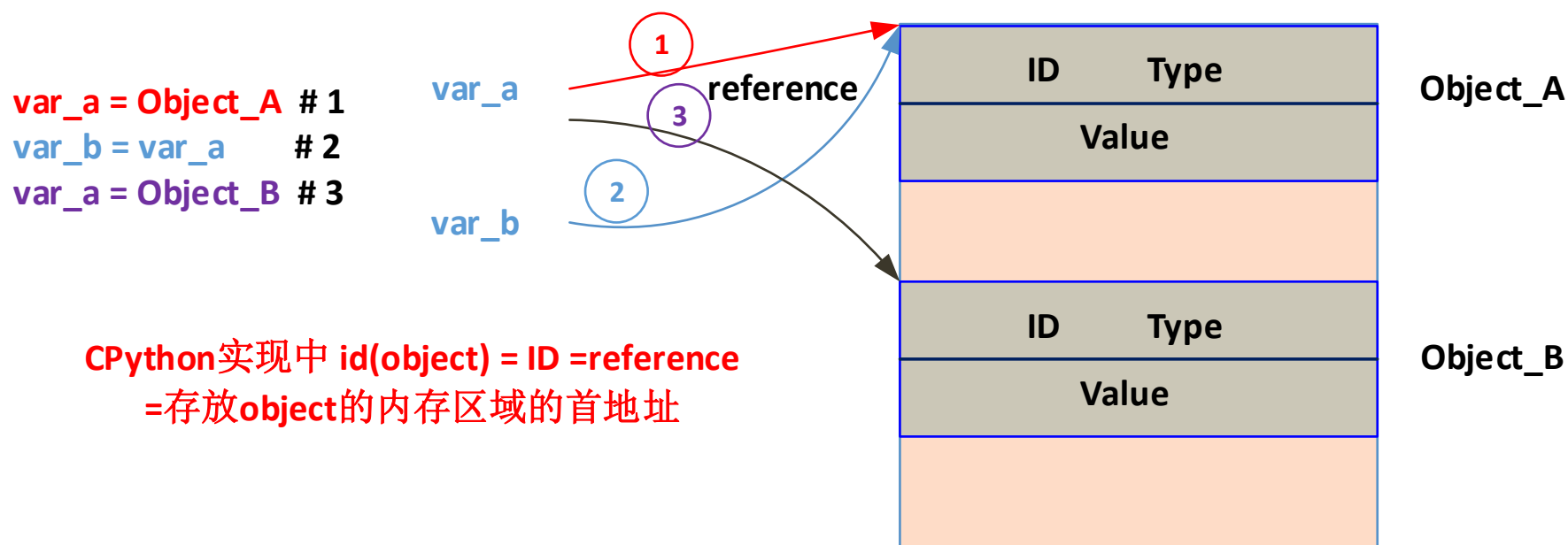
```
1
```

- >>>aList[1][2]

```
6
```

回顾：Python对象、变量与赋值

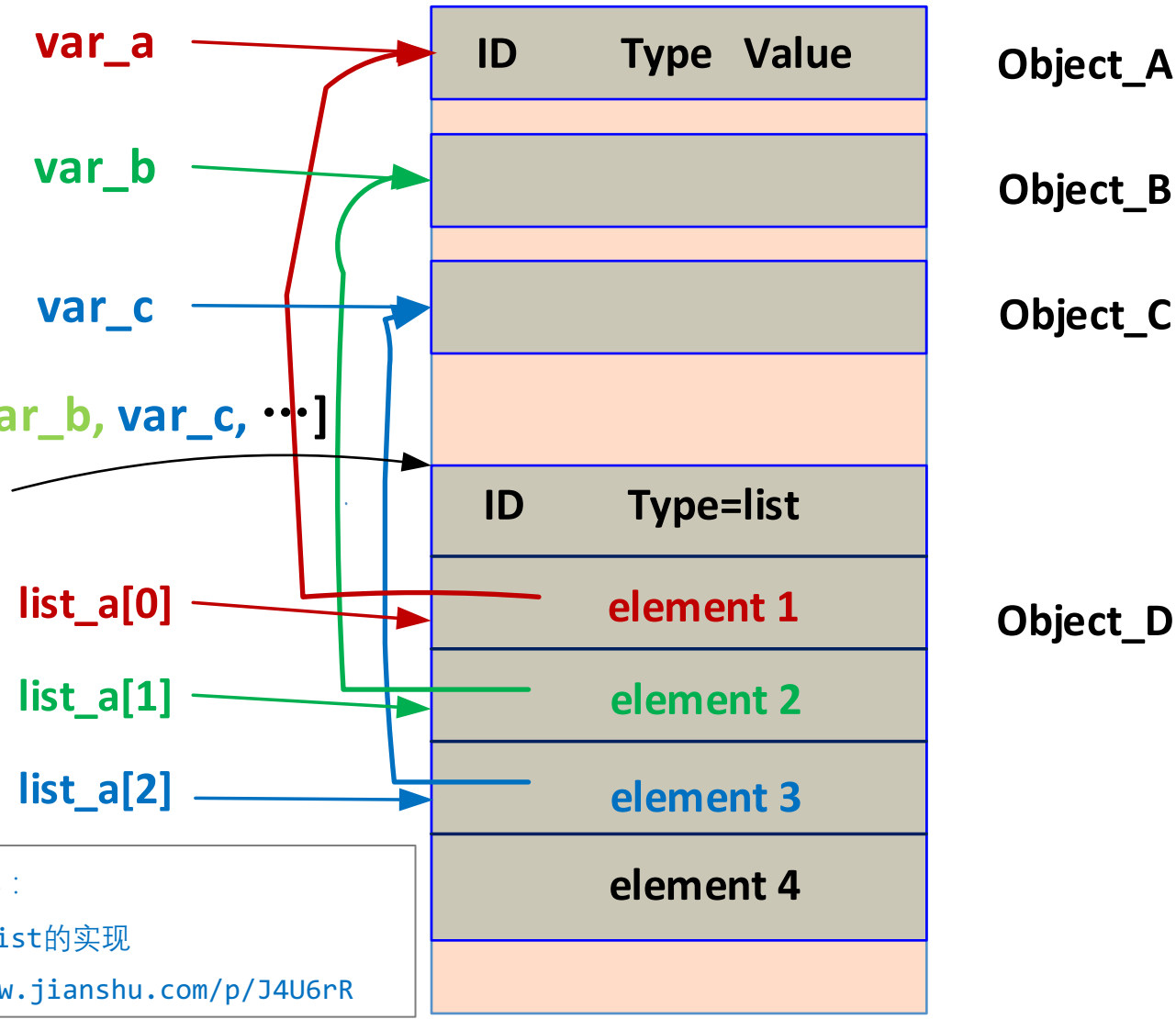
列表中的元素就是iterable对象中的各个元素
(即分别保存了那些元素的引用)



列表包含的是元素值（对象）的引用

```
>>> var_a = 1024
>>> var_b = "hello"
>>> var_c = [1,2]
>>> list_a=[var_a,var_b,var_c]
>>> list_a
[1024, 'hello', [1, 2]]
>>> id(list_a)
4443189704
>>> id(list_a[0])
4415353168
>>> id(var_a)
4415353168
>>> var_b is list_a[1]
True
```

list_a = [var_a, var_b, var_c, ...]



注意：

- 列表和列表元素是不同的对象。列表中包含的是元素值（对象）的引用，而不是直接包含元素值。从上述例子可以看出。
- 当列表元素增加或删除时，列表对象自动进行扩展或收缩内存，保证元素之间没有缝隙；（准确的说：元素对象的引用是连续存放的，例如图例的list_a[0]和list_a[1]，即使增删，也会保证元素对象的引用是连续存放的。）

2.1.1 列表创建与删除

- 也可以使用**list()函数**把元组、range对象、字符串、字典、集合或其他可迭代对象转换为列表。创建一个新的list对象，列表中的元素就是iterable对象中的各个元素（即分别保存了那些元素的引用）。

```
>>> list((3,5,7,9,11))           #将元组转换为列表
[3, 5, 7, 9, 11]
>>> list(range(1, 10, 2))        #将range对象转换为列表
[1, 3, 5, 7, 9]
>>> list('hello world')         #将字符串转换为列表
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> list({3,7,5})                #将集合转换为列表
[3, 5, 7]
>>> list({'a':3, 'b':9, 'c':78}) #将字典的“键”转换为列表
['a', 'c', 'b']
>>> list({'a':3, 'b':9, 'c':78}.items()) #将字典的“键:值”对转换为列表
[('b', 9), ('c', 78), ('a', 3)]
>>> x = list()                  #创建空列表

>>> x

[]
```

List()构造新的列表

- list(iterable)：创建一个新的对象，同时每个元素指向可迭代对象中各个元素指向的对象

```
list_a=[1,2,[-3,4]]
```

```
list_b = list(list_a)
```

```
list_b[0] = -1
```

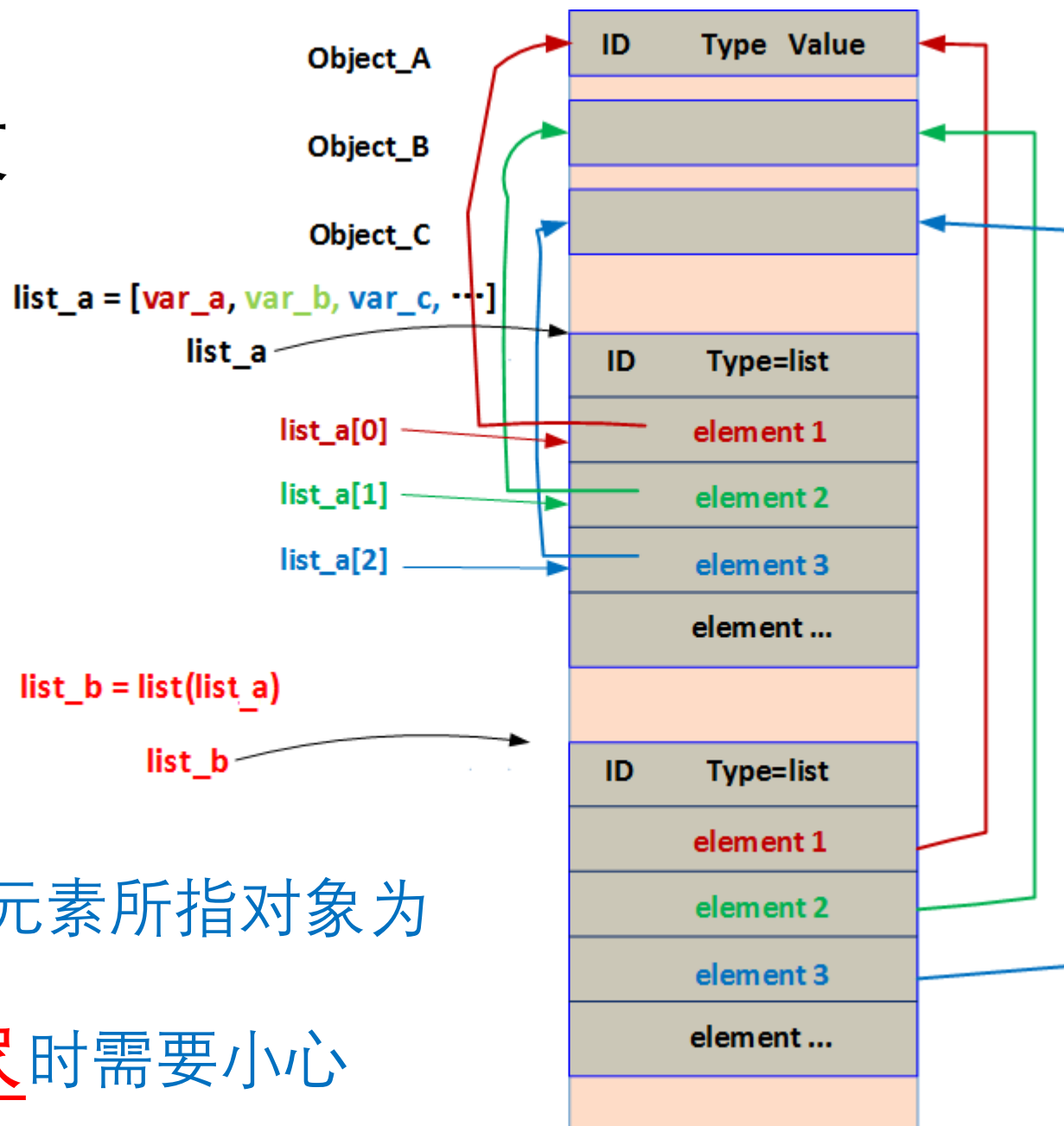
```
list_b[2][0] = 3
```

```
print(list_b)
```

```
>>> [-1, 2, [3, 4]]
```

```
print(list_a)
```

```
>>> [1, 2, [3, 4]]
```



在列表中的元素所指对象为

可变对象时需要小心

2.1.1 列表创建与删除

- 可以用list()函数将range对象转化为列表,上面的代码中用到了**内置函数range()**

```
>>> list(range(1, 10, 2))  
[1, 3, 5, 7, 9]
```

- **内置函数range()**是一个非常有用的函数, 后面会多次用到, 该函数语法为

range(stop)

range(start, stop[, step])

- 该函数在Python 3.x中[返回一个range可迭代对象](#)^{*}, 产生一系列的整数
- 参数start表示起始值 (默认为0)
- 参数stop表示终止值 (结果中不包括这个值)
- 参数step可选, 表示步长 (默认为1)

```
>>> x = range(10)  
  
>>> x  
range(0, 10)  
  
>>> x[2] #range也是一个有序序列  
2  
  
>>> list(range(10))  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

^{*} range对象的访问效率比转化后的list对象高

- Python2.x有range和xrange两个函数, range返回一个list, 而xrange返回一个iterable对象
- Python3的range相当于xrange

课堂练习：list()

#将字符串转换成列表

```
list1 = list("hello")  
print(list1)
```

#将元组转换成列表

```
tuple1 = ('Python', 'Java', 'C++',  
          'JavaScript')  
list2 = list(tuple1)  
print(list2)
```

#将字典转换成列表

```
dict1 = {'a':100, 'b':42, 'c':9}
```

```
list3 = list(dict1)  
print(list3)
```

#将区间转换成列表

```
range1 = range(1, 6)  
list4 = list(range1)  
print(list4)
```

#创建空列表

```
print(list())
```

```
['h', 'e', 'l', 'l', 'o']  
['Python', 'Java', 'C++', 'JavaScript']  
['a', 'b', 'c']  
[1, 2, 3, 4, 5]  
[]
```

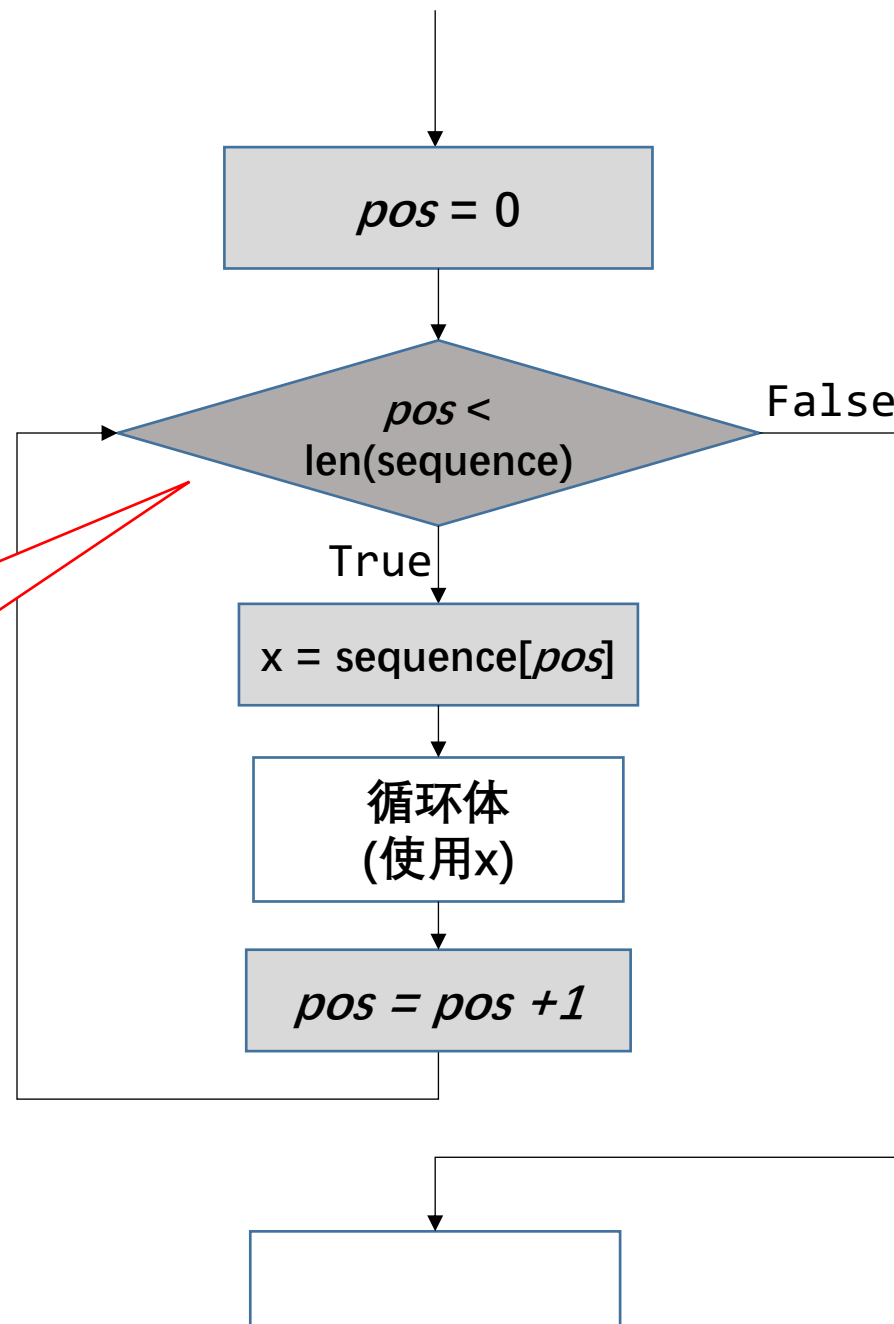
补充知识点

for循环

for循环流程(1)

for x in sequence:
循环体

1. `pos`表示内部隐含变量，灰色框表示内部隐含操作；
2. 每次比较时都是重新计算 `len(sequence)`；
3. `x`可以是任意合法变量名。



利用for循环流程遍历列表元素

```
monthNames = [ 'January', 'February', 'March',  
               'April', 'May', 'June', 'July', 'August',  
               'September', 'October', 'November', 'December' ]  
  
month = 1  
  
for name in monthNames:  
    print( "%2d月的英文是%s" % (month, name) )  
    month = month + 1
```

```
===== RESTART: (Python 3.6.4 Shell)  
1月的英文是January  
2月的英文是February  
3月的英文是March  
4月的英文是April  
5月的英文是May  
6月的英文是June  
7月的英文是July  
8月的英文是August  
9月的英文是September  
10月的英文是October  
11月的英文是November  
12月的英文是December  
>>>
```

```
monthNames = [ 'January', 'February', 'March',  
               'April', 'May', 'June', 'July', 'August',  
               'September', 'October', 'November', 'December' ]  
  
for index in range(12): #当然也可以用range对象  
    print("%2d月的英文是%s" % (index+1, monthNames[index]))
```



2.1 列表

- 列表元素的修改：

- 可以通过下标来修改序列中元素的值。
- del语句来删除元素
- 通过可变序列对象自身提供的方法来增加和删除元素。

```
del  
list_var[index]  
del list_var
```

```
>>> li=[1000,"123",[1,2]]  
>>> id(li[0])  
4524020720  
>>> li=[1000,"123",456,"789"]  
>>> id(li)  
50926472  
>>> li[0]=2000  
>>> id(li)  
50926472  
>>> li  
[2000, '123', 456, '789']  
>>> del li[0]  
>>> li  
['123', 456, '789']
```

注意：

如果是通过下标来修改序列中元素的值或通过可变序列对象自身提供的方法来增加和删除元素时，序列对象在内存中的起始地址是不变的，变的是对应的元素所指向对象或者序列对象的元素增删。

课堂练习：SHELL感受修改对id的影响

```
li=[1000,"123",[1,2]]
id(li[0])
li=[1000,"123",456,"789"]
id(li)
li[0]=2000
id(li)
id(li[0])
li
del li[0]
li
id(li)
```

逐行复制至SHELL环境，回车观察效果

2.1.1 列表创建与删除

- 当不再使用时，使用del命令删除整个列表，如果列表对象变量所指向的值（对象）不再有其他变量指向它，Python将同时删除该值（对象）。

```
>>> del a_list
```

```
>>> a_list
```

```
Traceback (most recent call last):
```

```
File "<pyshell#6>", line 1, in <module>
```

```
a_list
```

```
NameError: name 'a_list' is not defined
```

- 正如上面的代码所展示的一样，**删除列表对象变量之后，该变量就不存在了**，再次访问时将抛出异常“NameError”提示访问的变量名不存在。

列表方法

```
list_var[index]  
x in list_var  
x not in  
list_var
```

list
slicing

方法	说明
<code>list.append(x)</code>	将元素x添加至列表尾部
<code>list.extend(L)</code>	将列表L中所有元素添加至列表尾部
<code>list.insert(index, x)</code>	在列表指定位置index处添加元素x
<code>list.remove(x)</code>	在列表中删除首次出现的指定元素
<code>list.pop([index])</code>	删除并返回列表对象指定位置的元素
<code>list.clear()</code>	删除列表中所有元素，但保留列表对象，该方法在Python2中没有
<code>list.index(x)</code>	返回值为x的首个元素的下标
<code>list.count(x)</code>	返回指定元素x在列表中的出现次数
<code>list.reverse()</code>	对列表元素进行原地逆序
<code>list.sort()</code>	对列表元素进行原地排序
<code>list.copy()</code>	返回列表对象的浅拷贝，该方法在Python2中没有

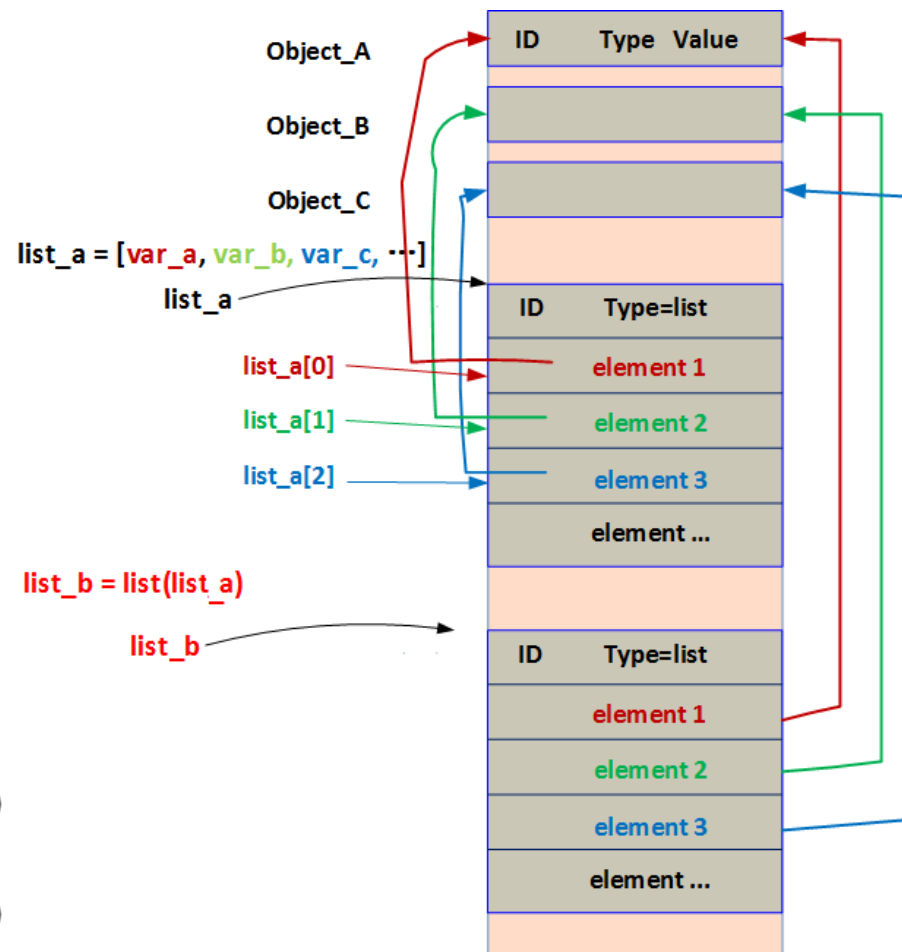
列表数据的复制

列表方法 `list.copy()` —— 浅拷贝

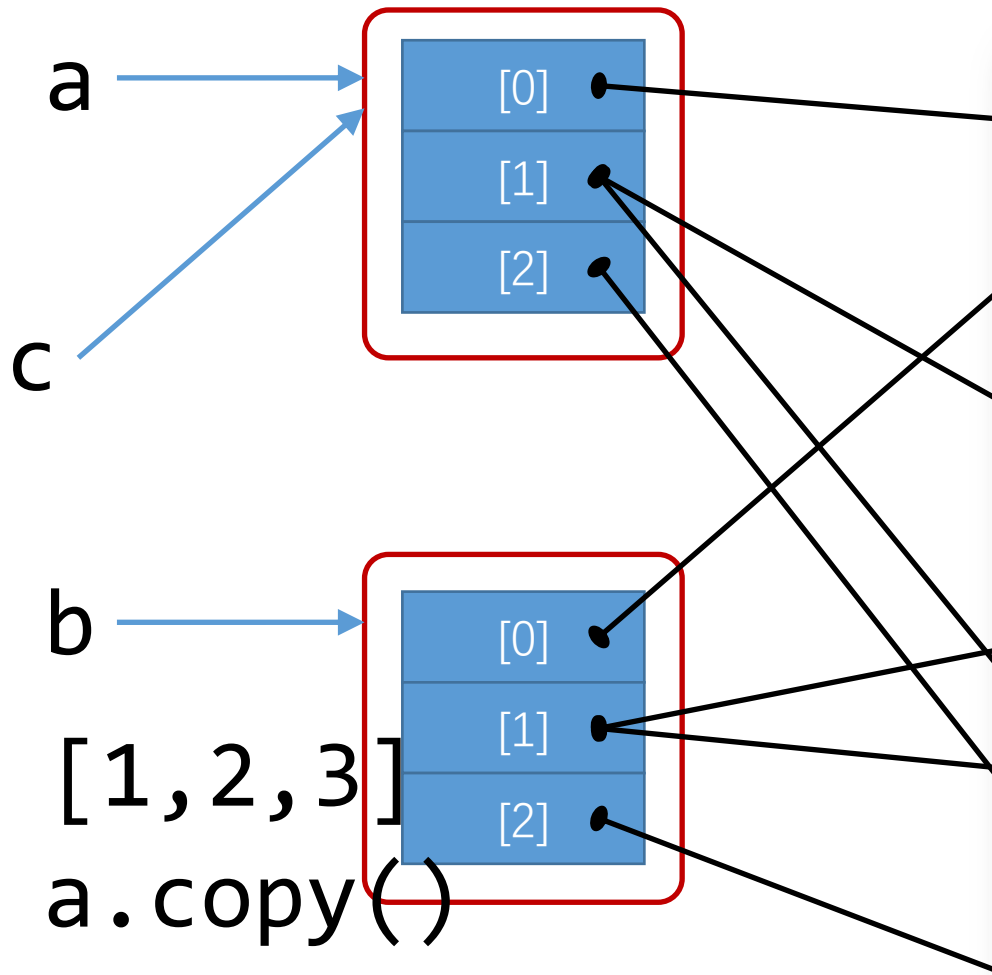
• 拷贝：

产生一个副本。

```
>>> a = [1024, 1036, 356]
>>> b = a.copy()
>>> a
[1024, 1036, 356]
>>> b
[1024, 1036, 356]
>>> id(a)
4565433800
>>> id(b)
4565433864
>>> id(a[0])
4565353424
>>> id(b[0])
4565353424
>>> id(a[1]) == id(b[1])
True
>>> id(a[2]) == id(b[2])
True
>>>
```



`a = [1, 2, 3]`
`b = a.copy()`
`b[1] = 4` # a不受影响
`c = a` #会发生什么?
`c[1] = 3` #会发生什么?

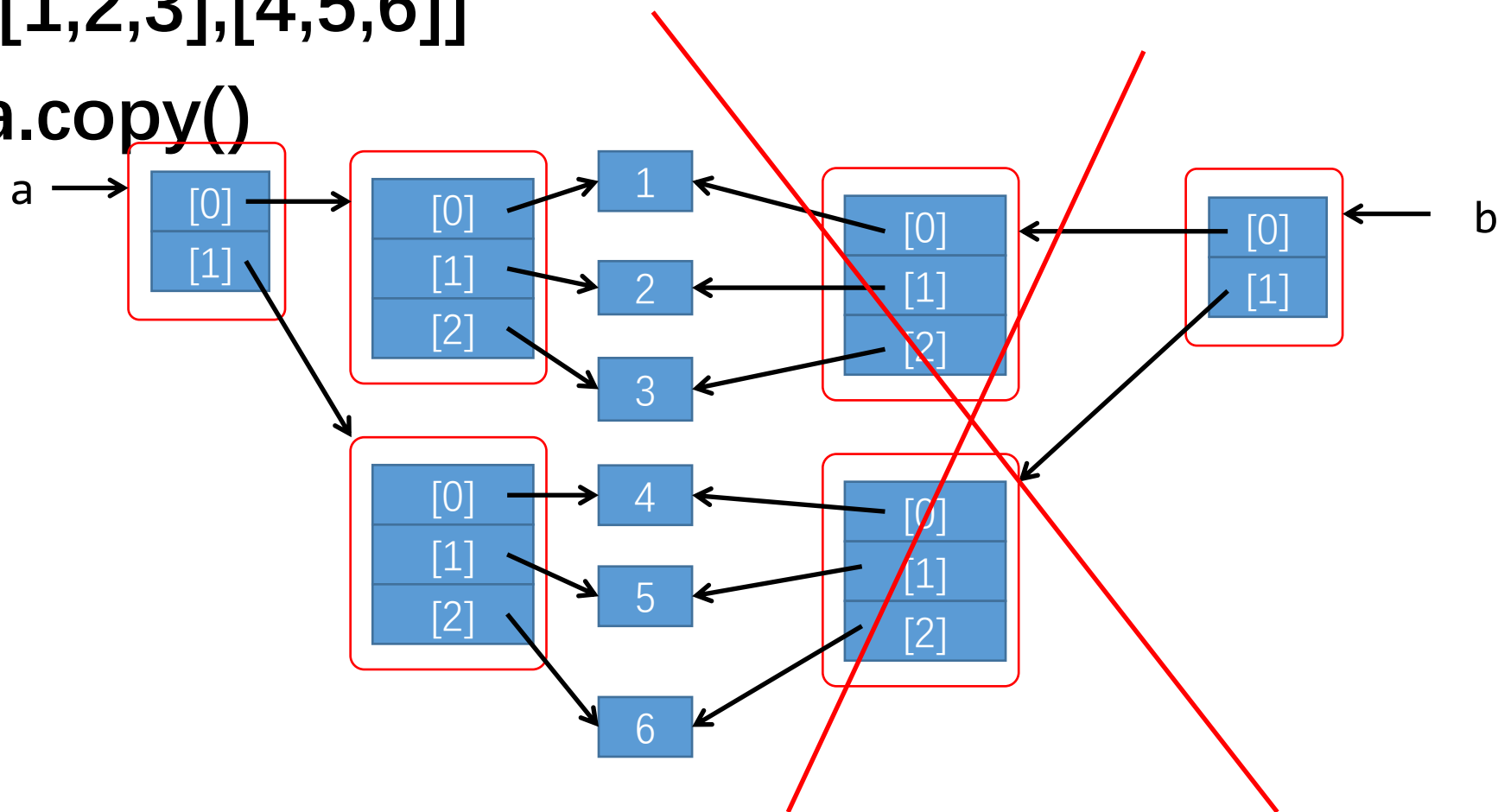


```
>>> a = [1, 2, 3]
>>> b = a.copy()
>>> b
[1, 2, 3]
>>> b[1] = 4
>>> a
[1, 2, 3]
>>> c = a
>>> c[1] = 3
>>> c
[1, 3, 3]
>>> a
[1, 3, 3]
```

拷贝 —— 制造完整的副本

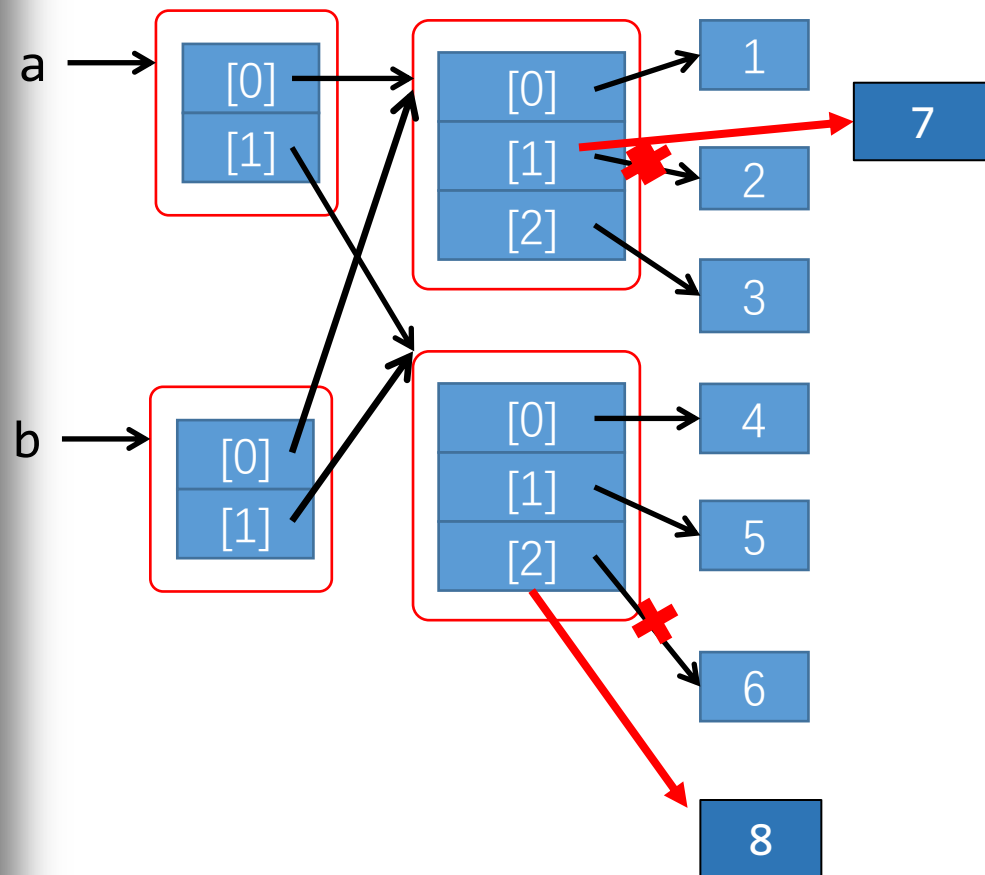
a = [[1,2,3],[4,5,6]]

b = a.copy()



浅拷贝 —— 只拷贝一层

```
>>> a = [[1,2,3],[4,5,6]]
>>> b = a.copy()
>>> b
[[1, 2, 3], [4, 5, 6]]
>>> b[0][1] = 7
>>> b
[[1, 7, 3], [4, 5, 6]]
>>> a # 修改列表b的元素影响了列表a
[[1, 7, 3], [4, 5, 6]]
>>> a[1][2] = 8
>>> a
[[1, 7, 3], [4, 5, 8]]
>>> b # 修改列表a的元素影响了列表b
[[1, 7, 3], [4, 5, 8]]
```



课堂练习：copy

```
a = [[1,2,3],[4,5,6]]
```

```
b = a.copy()
```

```
print(b)
```

```
b[0][1] = 7
```

```
print(a)
```

```
print(b)
```

```
a[1][2] = 8
```

```
print(a)
```

```
print(b)
```

```
a = [1,2,3,4,5,6]
```

```
b = a.copy()
```

```
print(b)
```

```
b[0] = 7
```

```
print(a)
```

```
print(b)
```

```
a[2] = 8
```

```
print(a)
```

```
print(b)
```

列表数据的增加

2.1.2 列表元素的增加

(1) 可以使用 “+” 运算符来实现将元素添加到列表中的功能。

- 虽然这种用法在形式上比较简单也容易理解，但严格意义上来讲，这并不是真的为列表添加元素，而是创建一个新列表，并将原列表中的元素和新元素依次复制到新列表的内存空间。
- 由于涉及大量元素的复制，该操作速度较慢，在涉及大量元素添加时不建议使用该方法。

```
>>> aList = [3,4,5]
```

```
>>> aList = aList + [7]
```

```
>>> aList
```

```
[3, 4, 5, 7]
```

+ 思考题：

请思考如何判定是创建了新列表？

2.1.2 列表元素的增加

(2) 使用列表对象的`append()`方法，**原地修改列表**，是真正意义上的在列表尾部添加元素，速度较快，也是推荐使用的方法。

```
>>> aList.append(9)
```

```
>>> aList
```

```
[3, 4, 5, 7, 9]
```

```
>>> aList.append([10])
```

```
[3, 4, 5, 7, 9,[10]]
```

+ 思考题：

请思考如何判定是原地修改列表？

2.1.2 列表元素的增加

(3) 使用列表对象的`extend()`方法可以将另一个迭代对象(tuple、str、list、range等)的所有元素添加至该列表对象尾部。通过`extend()`方法来增加列表元素也不改变列表对象的内存首地址，属于原地操作。

```
>>> a = [5, 2, 4]
>>> id(a)
54728008
>>> a.extend([7,8,9])
>>> a
[5, 2, 4, 7, 8, 9]
>>> id(a)
54728008
```

2.1.2 列表元素的增加

(4) 使用列表对象的`insert()`方法将元素添加至列表的指定位置。

语法：L.insert(index, object) -- insert object before index

```
>>> aList = [3, 4, 5, 7, 9]
```

```
>>> aList.insert(3,6)
```

```
>>> aList
```

```
[3, 4, 5, 6, 7, 9]
```

注意：

1. `insert()`方法会涉及到插入位置之后所有元素的移动，这会影响处理速度。
2. 列表删除方法`remove()`和`pop()`弹出非尾部元素时，也有类似问题。

建议：

除非有必要，否则应尽量避免在列表中间位置插入和删除元素的操作，而是优先考虑使用前面介绍的`append()`方法和下一小节介绍的`pop()`方法。

2.1.2 列表元素的增加

(5) 使用乘法来扩展列表对象，将列表与整数相乘，生成一个新列表，新列表是原列表中元素的重复。

重复元素实际上是多个元素指向同一对象

```
>>> aList = [1, 2, 3]
```

```
>>> aList = aList*3
```

```
>>> aList
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
>>> x = [[None] * 2] * 3
```

```
>>> x
```

```
[[None, None], [None, None],  
 [None, None]]
```

注意：

1. 该操作实际创建一个新列表，而不是原地修改。该操作同样适用于字符串、元组，并具有相同的特点。
2. 当使用 “*” 运算符将包含列表的列表重复并创建新列表时，注意到列表元素保存的是对象的引用，因此每个重复元素所指向对象（地址）是相同的。

3. 例如：

```
>>> x = [[1,2,3]]*3
```

```
>>> x
```

```
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
```

```
>>> x[0][0] = 10
```

```
>>> x
```

```
[[10, 2, 3], [10, 2, 3], [10, 2, 3]]
```

列表元素为可变

对象时需要特别注意

2.1.2 列表元素的增加

通过切片也可以实现元素的增加

- 各种列表元素增加方法的比较

#	列表元素增加方法	别名	效果
1	+	拼接	新建列表
2	append	附加	原地修改
3	extend	扩展	原地修改
4	insert	插入	原地修改
5	*	复制	新建列表

课堂练习：增加

- +
- append
- extend
- insert
- *

列表数据的删除

2.1.3 列表元素的删除

(1) 使用`del`命令删除列表中的指定位置上的元素。前面已经提到过，`del`命令也可以直接删除整个列表，这里不再赘述。

```
del list_obj[index]
```

```
>>> a_list = [3,5,7,9,11]
```

```
>>> del a_list[1]
```

```
>>> a_list
```

```
[3, 7, 9, 11]
```

下标不合法时，抛出异常：

`IndexError: list assignment index out of range`

2.1.3 列表元素的删除

(2) 使用列表的`pop([index])`方法删除并返回指定（默认为最后一个，即下标为-1或者len of list -1) 位置上的元素，如果给定的索引超出了列表的范围则抛出异常。

```
>>> a_list = list((3,5,7,9,11))
>>> a_list.pop()
11
>>> a_list
[3, 5, 7, 9]
>>> a_list.pop(1)
5
>>> a_list
[3, 7, 9]
```

```
>>> a_list.pop(10)
Traceback (most recent call last):
  File "<pyshell#85>", line 1, in
<module>
    a_list.pop(10)
IndexError: pop index out of range
```

2.1.3 列表元素的删除

(3) 使用列表对象的`remove(x)`方法删除首次出现的指定元素x, 即如果列表中有元素的值与x的值相等 (`list_var[index] == x`) , 则删除。如果列表中不存在要删除的元素, 则抛出异常 `ValueError`。

```
>>> a_list = [3,5,7,9,7,11]
>>> a_list.remove(7)
>>> a_list
[3, 5, 9, 7, 11]
```

```
>>> a=[1,2,3,[4,5],(6,7)]
>>> a.remove([4,5])
>>> a
[1, 2, 3, (6, 7)]
>>> a.remove(0)
Traceback (most recent call last):
  File "<pyshell#98>", line 1, in
<module>
    a.remove(0)
```

2.1.3 列表元素的删除

- 在使用循环进行列表元素删除时，需要使用正确的方法。例如大家会很自然地想到使用“循环+remove()”的方法删除列表中指定元素的所有重复。
- 例如，下面的代码成功地删除了列表中的重复元素，执行结果是完全正确的。

```
>>> x = [1,2,1,2,1,2,1,2,1]
>>> for i in x:
    if i == 1:
        x.remove(i)
>>> x
[2, 2, 2, 2]
```

```
def remove_all(list_, value):
    for item in list_:
        if item == value:
            list_.remove(value)

x = [1,2,1,2,1,2,1,2,1]
remove_all(x,1)
x = [1,2,1,2,1,1,1]
remove_all(x,1)
```

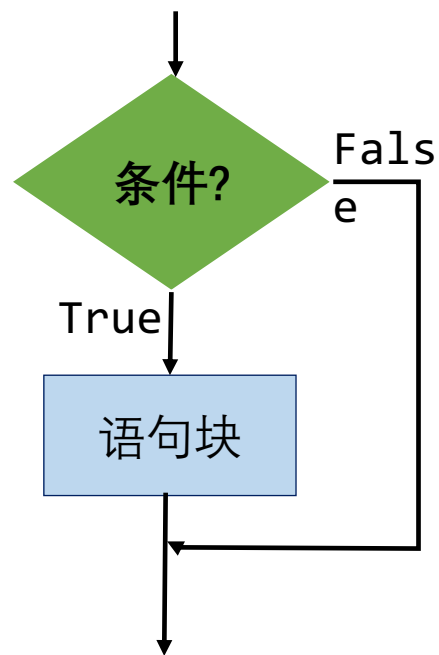
```
>>> x = [1, 2, 1, 2, 1, 2, 1, 2, 1]
>>> remove_all(x, 1)
>>> x
[2, 2, 2, 2]
>>> x = [1, 2, 1, 2, 1, 1, 1]
>>> remove_all(x, 1)
>>> x
[2, 2, 1]
```

补充知识点

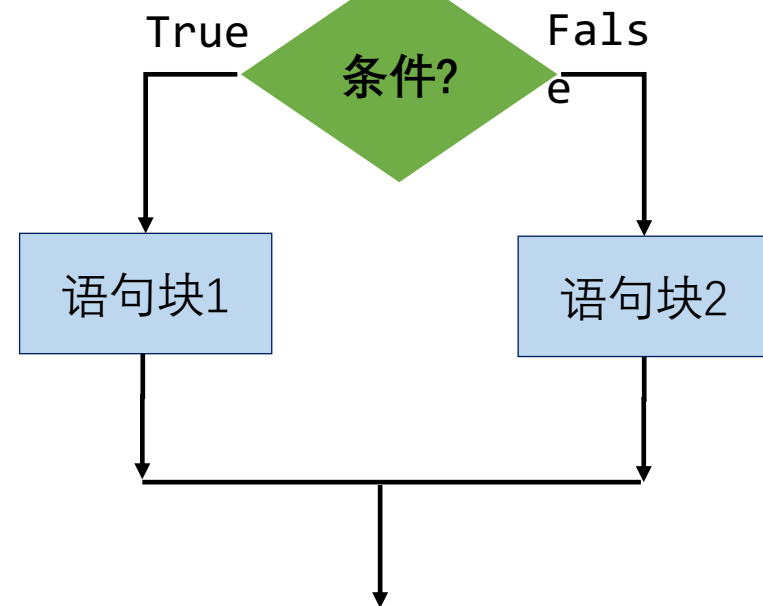
条件判断

3.2 选择结构

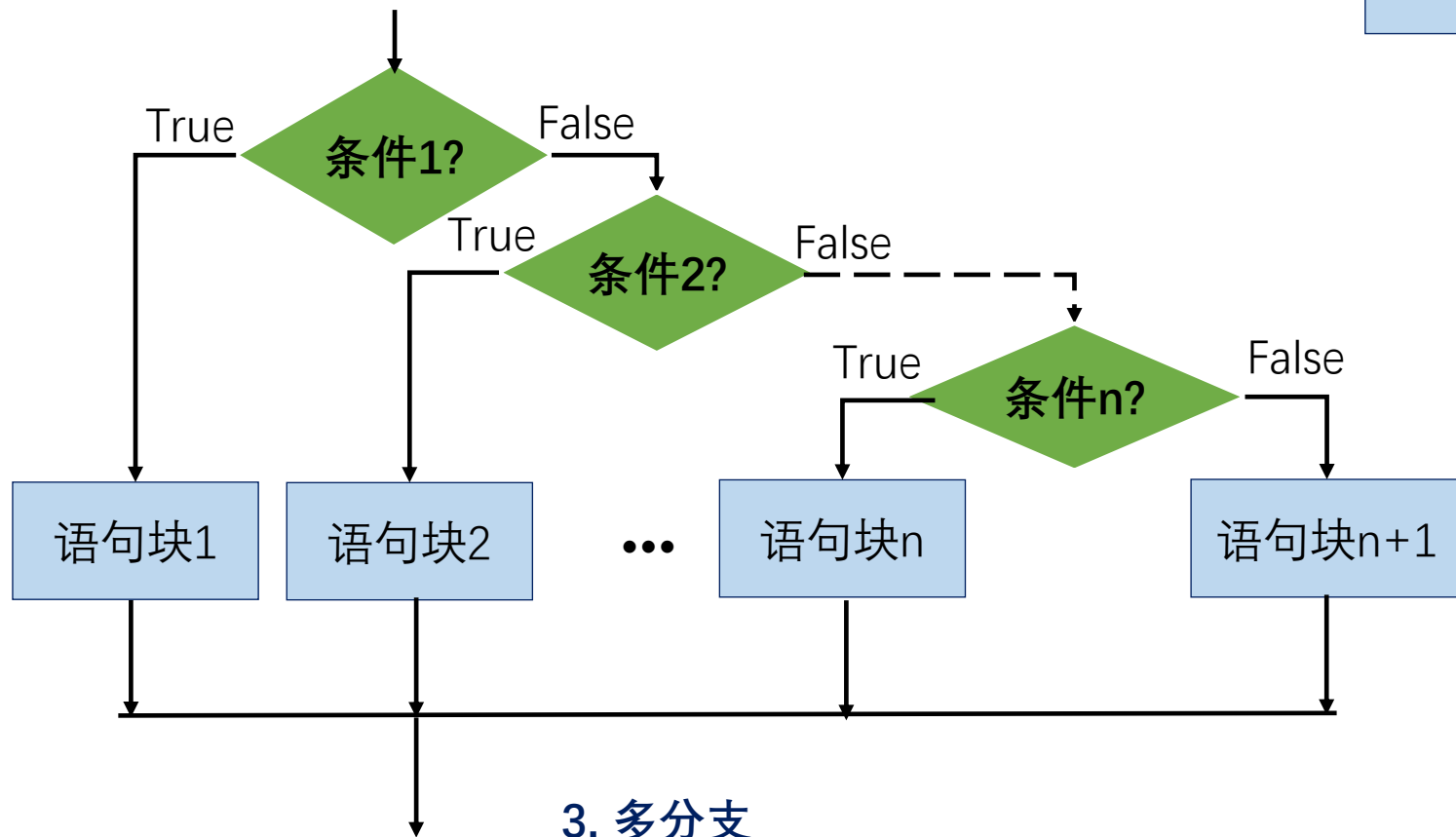
1. 单分支



2. 双分支



3. 多分支



3.2.1 单分支选择结构

if 条件表达式: # 其他语言要求条件表达式有括号
语句块

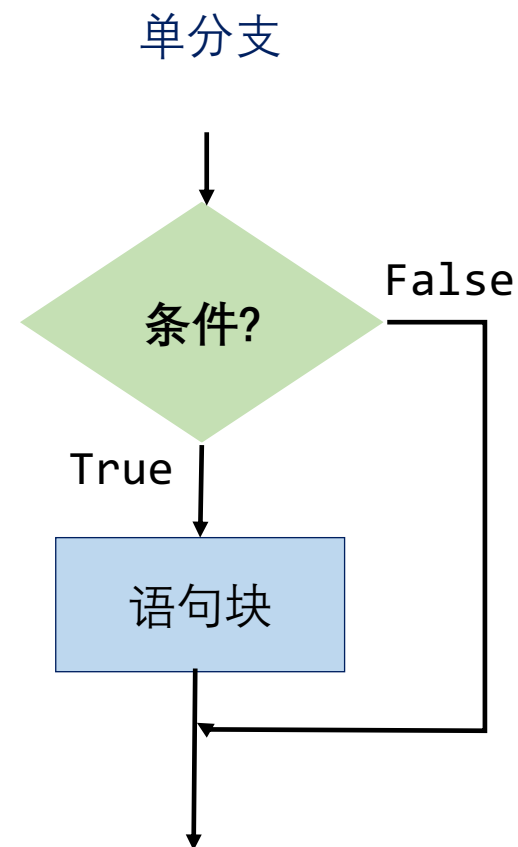
```
if item == value:  
    list_.remove(value)
```

提示用户输入两个数，然后按照从小到大顺序输出两个数

```
a, b = eval(input("请输入两个数（以英文逗号分隔）："))  
if a > b:  
    a, b = b, a  
print(a, b)
```

```
=====  
===== RESTART: C:/Users/zt  
请输入两个数（以英文逗号分隔）：123, 324  
123 324  
>>>  
=====  
===== RESTART: C:/Users/zt  
请输入两个数（以英文逗号分隔）：456, 123  
123 456  
>>>  
...
```

```
if __name__ == '__main__':  
    main()
```



3.2.2 双分支结构

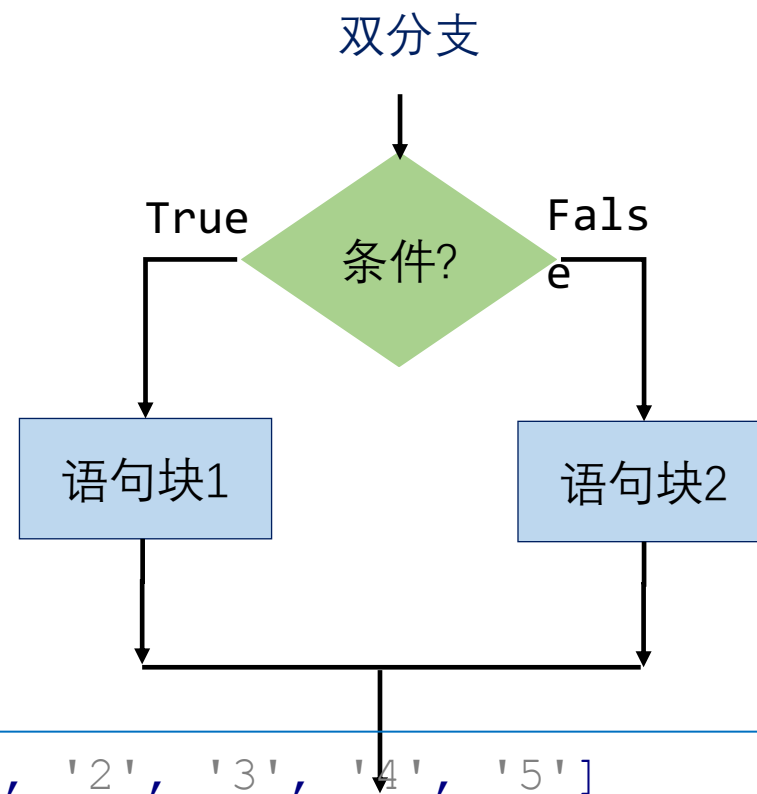
if 条件表达式:

语句块1

else:

语句块2

```
x = int(input("请输入整数:"))
if x % 2 == 0:
    print(x, '是一个偶数')
else:
    print(x, '是一个奇数')
```



```
chTest = ['1', '2', '3', '4', '5']
```

```
def test_nonempty(obj):
    if obj: #如果非空, 不需要 obj != []
        print(obj)
    else:
        print('Empty')
```

```
test_nonempty(chTest)
```

```
['1', '2', '3', '4', '5']
```

3.1 条件表达式

- **布尔 (bool)** 类型取值为**True**、**False**，分别对应整数1和0

```
>>> int(True), int(False)
```

```
(1, 0)
```

- 所谓条件表达式exp指的用在if/while等作为决定分支走向或循环结束条件时所使用的表达式，此时对该表达式进行真值判断：**bool(expr)**
 - 如果其值为**False(比较等运算结果)**、**0 (数值类型)**、**None**、**空序列对象**，则取值为**False**
 - 所谓空序列对象obj指的是其长度为0，即len(obj) == 0
"" [] () set() {} range(0)等取值False
 - 否则取值为True，即**非0**、**非空对象**等为**True**。
1 -5 "abc" [1,2] (1,) {one:'one'}

2.1.3 列表元素的删除

- 然而，上面这段代码的逻辑是错误的，尽管执行结果是正确的。如果所处理的数据发生了一点变化，同样的代码执行，然而当循环结束后却发现并没有把所有的“1”都删除，只是删除了一部分。

```
>>> x =  
[1,2,1,2,1,1,1]  
>>> for i in x:  
    if i ==  
1:  
    x.remove(i)  
>>> x  
[2, 2, 1]
```

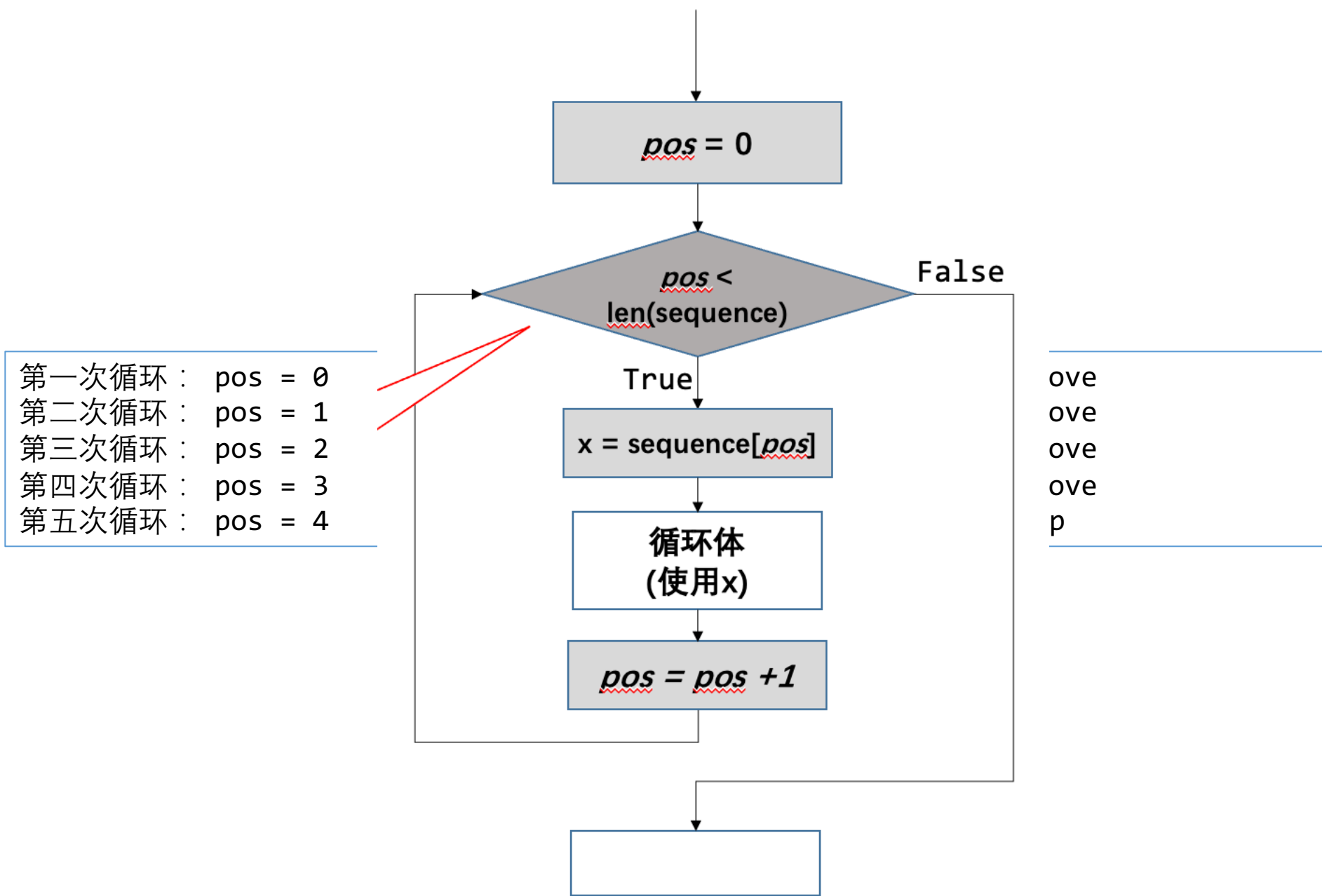
```
pos = 0    [1,2,1,2,1,1,1]    i = x[pos] → 1    remove  
pos = 1    [2,1,2,1,1,1]    i = x[pos] → 1    remove  
pos = 2    [2,2,1,1,1]    i = x[pos] → 1    remove  
pos = 3    [2,2,1,1]    i = x[pos] → 1    remove  
pos = 4    [2,2,1]    pos >= len(x)  
stop
```

为什么？

1. 列表 x 会随着元素被删除而变化，所以，每轮循环处理的 x 可能是不一样的。
2. 每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了。

怎么办？ 优先第一种

1. 在循环的判断条件部分，使用新的列表（如列表的切片）替代原始列表，这样即使原始列表因元素删除或增加而变化，作为循环判断的新列表不会变化。
2. 使用正确的顺序，例如从后往前依次判断。



2.1.3 列表元素的删除

- 正确的代码：

```
>>> x = [1,2,1,2,1,1,1]
```

```
>>> for i in list(x):
```

```
    if i == 1:
```

```
        x.remove(i)
```

或者：

```
>>> x = [1,2,1,2,1,1,1]
```

```
>>> for i in range(len(x)-1,-1,-1):
```

```
    if x[i]==1:
```

```
        del x[i]
```

列表推导式：`x = [i for i in x if i != 1]`（后面介绍）

在循环的判断条件部分，使用新列表替代原始列表，这样即使原始列表因元素删除或增加而变化，已经生成的列表不会变化。

1. `list(x)` 创建了新的列表，逐个从新列表中取出数据进行比较，但删除的是旧列表的数据，新列表不会有移位的问题

2. 想想可不可以用 `for i in x.copy()`

3. 以后还会学到切片 `x[::]`

使用正确的顺序，例如从后往前依次判断。

课堂练习：删除

- remove
- pop
- clear(自行尝试)

列表数据的读取/查询

2.1.4 列表元素访问与计数

- Review: (1) 使用**下标**直接访问列表元素

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> aList[3]
```

```
6
```

```
>>> aList[3] = 5.5
```

```
>>> aList
```

```
[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

如果指定下标不存在，则抛出异常

```
>>> aList[15]
```

Traceback (most recent call last):

File "<pyshell#34>", line 1, in <module>

aList[15]

IndexError: list index out of range

2.1.4 列表元素访问与计数

- (2) 使用列表对象的`index`方法获取指定值首次出现的下标

语法：L.index(value, [start, [stop]]) -> integer -- return first index of value.

Raises ValueError if the value is not present.

```
>>> aList
```

```
[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

```
>>> aList.index(7)
```

```
4
```

- 若列表对象中不存在指定值，则抛出异常

```
>>> aList.index(100)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#36>", line 1, in <module>
```

```
    aList.index(100)
```

```
ValueError: 100 is not in list
```

2.1.4 列表元素访问与计数

- 使用列表对象的**count方法**统计指定元素在列表对象中出现的次数

```
>>> aList  
[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]  
>>> aList.count(7)  
1  
>>> aList.count(0)  
0  
>>> aList.count(8)  
0
```

2.1.5 成员资格判断

- 判断列表中是否存在指定的值
 - (1) 可以使用前面介绍的`count()`方法，如果存在则返回大于0的数，如果返回0则表示不存在。
 - (2) 或者，使用更加简洁的“`in`”关键字来判断一个值是否存在于列表中，返回结果为“True”或“False”。关键字`in`也可用于其他可迭代对象，包括元组、字典、`range`对象、字符串、集合等。常在循环中使用关键字`in`对可迭代对象的元素进行遍历。

```
>>> aList
[3, 4, 5, 5.5, 7, 9, 11, 13,
15, 17]
>>> 3 in aList
True
>>> 18 in aList
False
```

```
>>> bList = [[1], [2], [3]]
>>> 3 in bList
False
>>> 3 not in bList
True
>>> [3] in bList
True
```

课堂练习：查询

- 下标与越界错误
- index
- count
- in

下节继续